

RESUM FUNCTOR, MONOIDE I MÒNADA

UT15



FUNCTOR

En la programació funcional un Functor ens serveix per a poder **aplicar una transformació a un valor que es troba dins d'un context** (mapping o mapeig).

Això ens permet treballar amb valors encapsulats i aplicar operacions sobre ells **sense haver de trencar la seua estructura original**, proporcionant una **abstracció potent** per a treballar amb dades de manera funcional.

Fixa't com s'utilitza la mateixa funció per a qualsevol estructura:

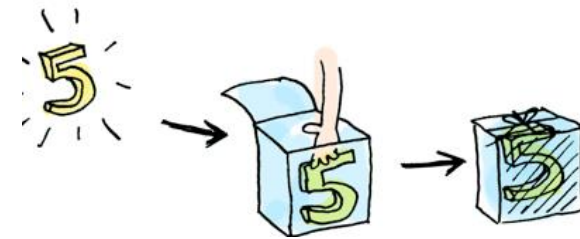
```
Functor<Integer> functor = Functor.of(3);      functor.map(x->x+3);  
Stream<Integer> flux= Stream.of(3);           flux.map(x->x+3);  
Optional<Integer> op = Optional.of(3);        op.map(x->x+3);
```

FUNCTOR - Mètodes importants

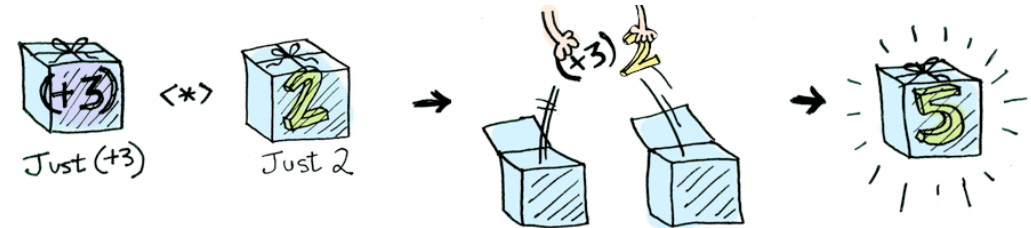
FUNCTOR -> map <\$>



POINTED FUNCTOR -> of



APPLICATIVE FUNCTOR -> ap <*>



MONOIDE

En la programació funcional els monoides ens permeten **encadenar operacions associatives** mitjançant una estructura algebraica que garanteix l'**associativitat i l'existència d'un element neutre**.

Això ens permet **combinar múltiples valors d'una mateixa classe en un resultat únic**. A través de l'ús de monoides, podem realitzar càlculs més complexos descomposant-los en passos més simples i combinant-los en un resultat final. Aquesta característica és fonamental en la programació funcional per a crear seqüències d'operacions.

```
Max<Double> m1 = Max.of(23.15); Max<Double> m2 = Max.of(23.2); Max<Double> m3 = Max.of(20.99);  
¿ m1.concat(m2).concat(m3); ? // Max[23.2]
```

MÒNADA

En la programació funcional, una Mònada ens permet **aplicar operacions al valor que està dins d'un context**, com un Functor, i també permet **encadenar operacions** com un Monoide de forma segura, ja que gestiona de manera automàtica el **tractament de valors absents o excepcionals**, així com **valors en contexts niuats** o anidats.

Podem treballar amb valors que es troben en capes o nivells d'anidament, com ara col·leccions d'elements dins de col·leccions, streams dins d'streams, opcionals dins d'opcionals...

Utilitzant la funció "**flatMap**" ens permet encadenar diverses operacions de forma segura i clara, independentment de la presència de valors absents, excepcions o contextos niuats.

```
public class Monad {  
    public static void main(String[] args) {  
        // Exemple 1: Treballant amb valors potencialment absents  
        Optional<String> nomOpcional = Optional.ofNullable(null);  
        String majusclesPunt = nomOpcional.map(String::toUpperCase).map(s → s.concat(".")).orElse("--SENSE NOM--");  
        System.out.println("Nom: " + majusclesPunt);  
  
        // Exemple 2: Treballant amb valors en un context niuat  
        Optional<Optional<String>> opcionalNiuat = Optional.of(Optional.of("Valor niuat"));  
        String valor = opcionalNiuat.flatMap(op → op.map(String::toUpperCase)).orElse("--VALOR INEXISTENT--");  
        System.out.println("Valor: " + valor); // Mostrerà "VALOR NIUAT"  
  
        // ¿Diferència?  
        Optional.of(5).map(Optional::of).map(Optional::of).ifPresent(System.out::println);  
        Optional.of(5).flatMap(Optional::of).flatMap(Optional::of).ifPresent(System.out::println);  
    }  
}
```

```
// Exemple 3: Concatenació d'operacions independentment del nidament que generen  
Optional<String> optionalValue = Optional.of("Hello");
```

```
Optional<String> result = optionalValue  
    .map(value → value + " World")  
    .flatMap(Monad::afegirExclamacio)  
    //.filter(s → s.contains("a"))  
    .flatMap(Monad::convertirAMajuscles);
```

```
System.out.println(result.orElse("--VALOR INEXISTENT")); //
```

```
}
```

```
private static Optional<String> afegirExclamacio(String value) {  
    return Optional.of(value + "!");  
}
```

```
private static Optional<String> convertirAMajuscles(String value) {  
    return Optional.of(value.toUpperCase());  
}
```